Chapitre 1 – Introduction à l'automatisation réseau avec Python

1. Automatisation réseau

L'automatisation réseau consiste à utiliser des scripts (souvent en Python) pour **gérer, configurer, surveiller** des périphériques réseau sans intervention manuelle.

Exemples d'automatisation :

- Se connecter à 50 routeurs et exécuter la même commande.
- Sauvegarder automatiquement les configurations chaque nuit.
- Détecter les pannes via API ou SNMP.
- Déployer une nouvelle configuration de sécurité sur plusieurs sites.

Avantages :

- Gain de temps.
- Réduction des erreurs humaines.
- Meilleure cohérence des configurations.

2. Python pour le réseau

a) Variables et types de données

```python
ip = "192.168.1.1"
port = 22
username = "admin"
password = "cisco123"

print(f"Connexion à {ip}:{port} avec {username}")
```

b) Listes et dictionnaires : utiles pour gérer plusieurs équipements

```
# Liste d'adresses IP
ips = ["192.168.1.1", "192.168.1.2", "192.168.1.3"]

# Dictionnaire pour stocker plusieurs infos d'un équipement
router = {
    "hostname": "R1",
    "ip": "192.168.1.1",
    "vendor": "Cisco"
}

print(router["hostname"], router["ip"])
```

c) Boucles

```
for ip in ips:
    print(f"Connexion à {ip}...")
```

d) Modules Python utiles pour le réseau

- `os` → gérer les fichiers, scripts locaux.
- `ipaddress` → manipuler adresses IPv4/IPv6.
- `paramiko` → SSH avec Python.
- `netmiko` → automatisation multi-constructeurs.
- `requests` → interagir avec des APIs REST.

Exemple simple avec ipaddress :

```
import ipaddress

network = ipaddress.ip_network("192.168.1.0/30")
for ip in network.hosts():
    print(ip)
```

Chapitre 2 – Python et le réseau

1. Objectifs

- Découvrir les bibliothèques Python utiles pour les communications réseau.
- Savoir manipuler des adresses IP.
- Comprendre comment se connecter en SSH à un équipement.
- Créer ses premiers scripts d'interaction réseau.

2. La bibliothèque socket – communications réseau de base

Le module socket permet de créer des connexions réseau (TCP/UDP).

❖ Exemple : tester si un port est ouvert

Utile pour vérifier si les équipements sont joignables avant d'automatiser.

```python
import socket

def check_port(ip, port):
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.settimeout(2)  # Timeout de 2s
    try:
        sock.connect((ip, port))
        print(f"[OK] {ip}:{port} est ouvert")
    except:
        print(f"[ERREUR] {ip}:{port} est fermé/inaccessible")
    finally:
        sock.close()

# Test sur un routeur (ex: port SSH 22)
check_port("192.168.1.1", 22)
```

3. La bibliothèque ipaddress – gestion des IP

Manipulation d'adresses et sous-réseaux.

```python
import ipaddress

# Créer un réseau
net = ipaddress.ip_network("192.168.1.0/29")

print("Adresses disponibles :")
for host in net.hosts():
    print(host)
```

4. Connexion SSH avec paramiko

Permet de se connecter en SSH à un routeur/switch pour exécuter des commandes.

Exemple : Résultat attendu : sortie de la commande show version (ou équivalent selon le type d'équipement).

```python
import paramiko

ip = "192.168.1.1"
username = "admin"
password = "cisco123"

# Création du client SSH
ssh = paramiko.SSHClient()
ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())

try:
    ssh.connect(ip, username=username, password=password)
    print(f"[OK] Connexion établie avec {ip}")

    # Exécuter une commande
    stdin, stdout, stderr = ssh.exec_command("show version")
    print(stdout.read().decode())

except Exception as e:
    print(f"[ERREUR] Impossible de se connecter : {e}")
finally:
    ssh.close()
```

5. Script multi-équipements

Connexion SSH sur une liste d'adresses IP.

```
devices = [
    {"ip": "192.168.1.1", "user": "admin", "pwd": "cisco123"},
    {"ip": "192.168.1.2", "user": "admin", "pwd": "juniper123"}
]

for dev in devices:
    try:
        ssh = paramiko.SSHClient()
        ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
        ssh.connect(dev["ip"], username=dev["user"], password=dev["pwd"])
        print(f"[OK] Connecté à {dev['ip']}")
        ssh.close()
    except:
        print(f"[ERREUR] Connexion à {dev['ip']} impossible")
```

6. Exercices pratique

a) Créer un script qui prend une liste d'IP.

b) Pour chaque IP, vérifier si le port **22 (SSH)** est ouvert avec socket.

c) Si le port est ouvert, tenter une connexion SSH avec paramiko.

d) Exécuter la commande show ip interface brief (Cisco) ou ifconfig (Linux).

e) Sauvegarder les sorties dans un fichier texte.

Chapitre 3 - Automatisation avec Netmiko

1. Objectifs du chapitre

- Découvrir Netmiko, une bibliothèque Python dédiée aux équipements réseau.
- Se connecter automatiquement à différents constructeurs (Cisco, Juniper, Huawei, Mikrotik…).
- Exécuter des commandes en masse.
- Sauvegarder des configurations.

2. Netmiko

- Bibliothèque Python développée par Kirk Byers.
- Basée sur Paramiko, mais optimisée pour les périphériques réseau.
- Supporte + de 70 plateformes (Cisco IOS, NX-OS, Juniper JunOS, HP, Huawei, Mikrotik, Arista…).
- Fournit des méthodes simples :

  send_command() → exécuter une commande show.

  send_config_set() → envoyer des configurations.

Installation via :

```
pip install netmiko
```

3. Connexion simple avec Netmiko

Le code Python suivant utilise la bibliothèque **Netmiko** pour automatiser la gestion d'un équipement réseau. Il commence par définir les paramètres de connexion — tels que le type d'équipement, son adresse IP et les identifiants — dans un dictionnaire nommé device. Ensuite, il établit une connexion sécurisée (SSH par défaut) avec l'équipement en utilisant la classe ConnectHandler et transmet le dictionnaire comme arguments. Une fois la connexion établie, il envoie la commande CLI "show ip interface brief", récupère la sortie complète de l'équipement, l'affiche dans la console, puis ferme la session proprement pour libérer les ressources.

```python
from netmiko import ConnectHandler

device = {
    "device_type": "cisco_ios",   # Type d'équipement
    "ip": "192.168.1.1",
    "username": "admin",
    "password": "cisco123"
}

# Connexion
net_connect = ConnectHandler(**device)

# Exécuter une commande
output = net_connect.send_command("show ip interface brief")
print(output)

# Fermer la connexion
net_connect.disconnect()
```

4. Automatisation multi-équipements

Le code Python ci-dessous utilise la bibliothèque **Netmiko** pour automatiser la connexion et l'exécution de commandes sur plusieurs équipements réseau de manière séquentielle. Il commence par une liste de dictionnaires, où chaque dictionnaire contient les paramètres de connexion spécifiques à un appareil (type, adresse IP, identifiants). Le script parcourt ensuite cette liste à l'aide d'une boucle for. À chaque itération, il tente d'établir une connexion sécurisée, exécute la commande "show version" et affiche la sortie. Un bloc try...except est utilisé pour gérer les erreurs, comme une adresse IP inaccessible ou des identifiants incorrects, garantissant que le script ne s'arrête pas en cas de défaillance d'un seul appareil et continue de traiter les suivants.

```python
from netmiko import ConnectHandler

devices = [
    {"device_type": "cisco_ios", "ip": "192.168.1.1", "username": "admin", "password": "cisco123"},
    {"device_type": "juniper", "ip": "192.168.1.2", "username": "admin", "password": "juniper123"},
]

for dev in devices:
    try:
        conn = ConnectHandler(**dev)
        print(f"[OK] Connecté à {dev['ip']}")
        output = conn.send_command("show version")
        print(output)
        conn.disconnect()
    except Exception as e:
        print(f"[ERREUR] Connexion {dev['ip']} : {e}")
```

5. Envoyer des configurations et sauvegarde

Dans cet exemple on active une nouvelle interface

```
config_commands = [
    "interface GigabitEthernet0/1",
    "description Lien vers WAN",
    "ip address 192.168.10.1 255.255.255.0",
    "no shutdown"
]

output = net_connect.send_config_set(config_commands)
print(output)
```

Dans la suivante on sauvegarde la configuration dans un fichier backup_R1.txt

```
output = net_connect.send_command("show running-config")
with open("backup_R1.txt", "w") as f:
    f.write(output)
print("Sauvegarde effectuée")
```

6. Exercice pratique

- Créer une liste de 3 périphériques (Cisco, Juniper, Mikrotik).
- Écrire un script qui :
  - Se connecte en SSH à chaque périphérique via Netmiko.
  - Exécute show version et show ip interface brief.
  - Sauvegarde les résultats dans des fichiers séparés (backup_R1.txt, backup_R2.txt, …).

Chapitre 4 - APIs et RESTCONF

1. **Objectif du chapitre**
   - Comprendre ce qu'est une API et pourquoi elles sont utilisées dans le réseau.
   - Découvrir **RESTCONF**, standard utilisé par Cisco/Juniper pour l'automatisation.
   - Utiliser Python (requests) pour interagir avec des équipements via API.
   - Récupérer et modifier des configurations en JSON/XML.
2. **Objectif du chapitre**

Afin de tester les fonctionnalites dans ce chapitre vous devez installer GNS3 :

- Pour les codes utilisant **Netmiko**, la plupart des images Cisco IOS, Juniper vMX ou Arista vEOS disponibles dans GNS3 fonctionneront parfaitement, car elles supportent toutes les connexions SSH/CLI.
- Pour les codes utilisant **RESTCONF**, vous devez utiliser des images plus récentes qui supportent les API, comme **Cisco IOS-XE (CSR 1000v)** ou **Cisco NX-OS (Nexus 9000v)**.

Lien : https://gns3.com/marketplace/appliances/cisco-nx-osv-9000

3. **API réseau**

Une **API (Application Programming Interface)** permet à un programme de dialoguer avec un périphérique réseau via des requêtes HTTP (GET, POST, PUT, DELETE).

- **REST API** : basée sur HTTP + JSON/XML.
- **RESTCONF** : extension normalisée pour les équipements réseau, basée sur YANG.
- Avantage : pas besoin de SSH → tout passe par HTTP(S).

Installation via :

```
pip install requests
```

Exemple basique :

```python
import requests

url = "https://jsonplaceholder.typicode.com/posts/1"
response = requests.get(url)

print("Code HTTP :", response.status_code)
print("Contenu :", response.json())
```

Ce code Python utilise la bibliothèque **requests** pour interagir avec une API web, une pratique courante en automatisation et en développement logiciel. Il envoie une requête GET à l'URL spécifiée pour récupérer des données au format JSON, puis il affiche le code de statut HTTP pour vérifier si la requête a réussi et le contenu de la réponse. La méthode .json() convertit automatiquement la réponse de l'API en un dictionnaire Python, rendant les données faciles à manipuler.

4. Connexion à un équipement via RESTCONF (Cisco IOS XE)

```python
import requests
from requests.auth import HTTPBasicAuth

# Désactiver les warnings SSL (pour lab)
requests.packages.urllib3.disable_warnings()

# Informations du périphérique
url = "https://192.168.1.10/restconf/data/ietf-interfaces:interfaces"
headers = {"Accept": "application/yang-data+json"}
auth = HTTPBasicAuth("admin", "cisco123")

# Requête GET
resp = requests.get(url, headers=headers, auth=auth, verify=False)

if resp.status_code == 200:
    interfaces = resp.json()
    print("Interfaces détectées :", interfaces)
else:
    print(f"Erreur {resp.status_code}")
```

Ce code Python utilise la bibliothèque **requests** pour interagir avec une API **RESTCONF** d'un équipement réseau. Il est conçu pour récupérer automatiquement des données sur les interfaces de l'appareil de manière structurée. Le script définit l'URL de l'API pour accéder au modèle de données standardisé ietf-interfaces, spécifie dans les en-têtes qu'il attend une réponse au format

JSON et fournit des identifiants d'utilisateur et de mot de passe pour l'authentification HTTP. Il envoie ensuite une requête GET et, si la réponse est 200 (succès), il convertit les données JSON en un dictionnaire Python pour les afficher. Si un autre code de statut est renvoyé, cela signale une erreur (par exemple, des identifiants incorrects).

5. Modifier une configuration via API

❖ Ajout d'une description à une interface :

```
import json

url = "https://192.168.1.10/restconf/data/ietf-interfaces:interfaces/interface=GigabitEthernet1"

payload = {
    "ietf-interfaces:interface": {
        "name": "GigabitEthernet1",
        "description": "Interface configurée via RESTCONF",
        "type": "iana-if-type:ethernetCsmacd",
        "enabled": True
    }
}

headers = {
    "Content-Type": "application/yang-data+json",
    "Accept": "application/yang-data+json"
}

resp = requests.put(url, headers=headers, auth=auth, data=json.dumps(payload), verify=False)

print("Code HTTP :", resp.status_code)
```

Retour attendu : 200 OK si la modification a été appliquée.

❖ Exemple multi-équipements

```
devices = [
    {"ip": "192.168.1.10", "user": "admin", "pwd": "cisco123"},
    {"ip": "192.168.1.11", "user": "admin", "pwd": "cisco123"}
]

for dev in devices:
    url = f"https://{dev['ip']}/restconf/data/ietf-interfaces:interfaces"
    resp = requests.get(url, headers=headers, auth=HTTPBasicAuth(dev["user"], dev["pwd"]), verify=False)
    print(f"{dev['ip']} → {resp.status_code}")
```

Chapitre 5 – Outils avancés (NAPALM, Ansible, Monitoring avec Python).

1. **Objectifs**
   - Découvrir NAPALM, une bibliothèque Python multi-vendeurs pour l'automatisation.
   - Comprendre l'intégration avec Ansible.
   - Faire du monitoring réseau en temps réel avec Python.
2. **NAPALM : Présentation**

NAPALM (Network Automation and Programmability Abstraction Layer with Multivendor support):

Il Permet de gérer plusieurs équipements (Cisco, Juniper, Arista, etc.) avec une même API Python.

Installation via :

```
pip install napalm
```

Exemple :

```
from napalm import get_network_driver

driver = get_network_driver("ios")  # pour Cisco IOS
device = driver(hostname="192.168.1.10", username="admin", password="cisco123")

device.open()
print("Facts:", device.get_facts())   # Informations générales
print("Interfaces:", device.get_interfaces())  # Interfaces
device.close()
```

3. **NAPALM : Présentation**

Le code suivant utilise la bibliothèque **NAPALM** pour configurer un équipement réseau de manière sécurisée et interactive. Le script commence par définir la configuration souhaitée pour une interface (GigabitEthernet1) dans une variable. Ensuite, il ouvre une connexion vers l'appareil

et utilise device.load_merge_candidate() pour charger cette configuration dans un espace de "configuration candidate".

Avant d'appliquer les changements, la méthode device.compare_config() génère et affiche un **diff** qui montre précisément les lignes qui seront ajoutées ou supprimées. L'utilisateur est alors invité à confirmer l'application des changements. Si la réponse est "oui", les modifications sont validées sur l'appareil via device.commit_config(); sinon, elles sont annulées par device.discard_config(). Cette approche met en évidence le paradigme sécurisé de NAPALM, qui permet de vérifier les changements avant leur application, réduisant ainsi le risque d'erreurs.

```
config = """
interface GigabitEthernet1
 description Configuré avec NAPALM
 ip address 192.168.10.1 255.255.255.0
 no shutdown
"""

device.open()
device.load_merge_candidate(config=config)
print("Diff :", device.compare_config())

choice = input("Appliquer ? (y/n) : ")
if choice == "y":
    device.commit_config()
else:
    device.discard_config()

device.close()
```

4. Automatisation avec Ansible

Ansible est un outil d'automatisation (basé sur YAML + SSH/API).

- Avantage : déploiement massif sur des centaines d'équipements.
- Python peut être utilisé pour générer les fichiers d'inventaire ou les playbooks.

Exemple de playbook Ansible pour configurer une interface :

```yaml
- name: Configure Cisco IOS interface
  hosts: routers
  gather_facts: no
  tasks:
    - ios_config:
        lines:
          - description Configuré avec Ansible
        parents: interface GigabitEthernet1
```

5. Monitoring réseau avec Python

On peut faire du monitoring SNMP, API ou temps réel.

❖ Exemple ping monitoring

```python
import os
import time

hosts = ["192.168.1.1", "192.168.1.10", "192.168.1.20"]

while True:
    for host in hosts:
        response = os.system(f"ping -c 1 {host} > /dev/null 2>&1")
        status = "UP" if response == 0 else "DOWN"
        print(f"{host} → {status}")
    time.sleep(5)
```

❖ Exemple SNMP (avec pysnmp)

Ce code Python utilise la bibliothèque PySNMP pour interroger un équipement réseau via le protocole SNMP (Simple Network Management Protocol). Le script tente d'établir une connexion à l'adresse IP 192.168.1.1 sur le port par défaut 161 en utilisant la chaîne de communauté public. Il demande une information spécifique : le descripteur système (sysDescr), identifié par son OID (Object Identifier) 1.3.6.1.2.1.1.1.0. Si la requête réussit, il affiche la valeur renvoyée par l'appareil,

qui est généralement une description du matériel ou du logiciel. Si une erreur de connexion ou de protocole se produit, le script affiche un message d'erreur.

```
from pysnmp.hlapi import *

for (errorIndication, errorStatus, errorIndex, varBinds) in getCmd(
    SnmpEngine(),
    CommunityData('public'),
    UdpTransportTarget(('192.168.1.1', 161)),
    ContextData(),
    ObjectType(ObjectIdentity('1.3.6.1.2.1.1.1.0'))  # sysDescr
):
    if errorIndication:
        print(errorIndication)
    else:
        for varBind in varBinds:
            print(varBind)
```

6. **Exercice pratique**

   Créer un script avec **NAPALM** qui :

   - Récupère les facts (hostname, vendor, uptime).
   - Récupère les interfaces actives.
   - Sauvegarde la configuration dans un fichier.

   Mettre en place un script **de monitoring ping** sur une liste de routeurs.

   Intégrer un petit dashboard avec **Matplotlib** ou **Plotly** pour afficher l'état réseau.

Chapitre 6 - Projet final : Application d'automatisation réseau complète avec Python

Créer une application complète capable de :

1. Découvrir les équipements d'un réseau.
2. Récupérer des données (interfaces, uptime, traffic, configuration).
3. Appliquer des configurations automatiquement.
4. Faire du monitoring et générer des rapports ou dashboards.

Architecture du projet :

```
NetworkAutomationApp/
│
├─ main.py           # Script principal
├─ devices.yaml      # Liste des équipements
├─ modules/
│   ├─ discovery.py  # Découverte réseau
│   ├─ napalm_utils.py # Connexion, lecture et configuration
│   ├─ monitoring.py   # Ping/SNMP monitoring
│   ├─ reports.py      # Génération rapports/graphes
│
└─ dashboards/
    └─ plotly_dash_app.py # Dashboard en temps réel
```